

Retrieval-Augmented Generation

Speaker
Arman Grigoryan

RAG

retrieval + generation
+ product judgement

The uncomfortable truth

Most AI demos are easy. Most AI products fail on messy company data.

LLM alone

- Confident hallucinations
- No private data
- Knowledge gets stale
- Too expensive

Prototype trap

- Looks magical in demo
- Breaks on edge cases
- No eval dataset
- Unknown cost per answer

Product-ready AI

- Grounded answers
- Fresh sources
- Measurable quality
- User trust loop

INTERACTIVE
POLL

Where would wrong AI answers cost real money in your startup?

RAG is more than prompt engineering

Prompting is a message. RAG is a system.

Prompt Engineering

- Static instructions
- Manual context copy-paste
- Good for demos
- Hard to update systematically

RAG System

- Live retrieval layer
- Permissions + metadata
- Citations and traceability
- Evaluated like a product

Do not ask “which model?” first. Ask “what evidence should the model see?”

RAG vs fine-tuning: decision matrix

Use the cheapest lever that fixes the actual failure.

Problem	Use RAG	Use Fine-tuning
Needs private/latest facts	YES — retrieve facts	No — weights go stale
Wrong tone or format	Maybe	YES — teach style
Domain-specific reasoning	Maybe with examples	Maybe with evals
Need source citations	YES	No
FAQ with school behavior + 1mln QA	10 minutes to retrieve	<.0001 sec

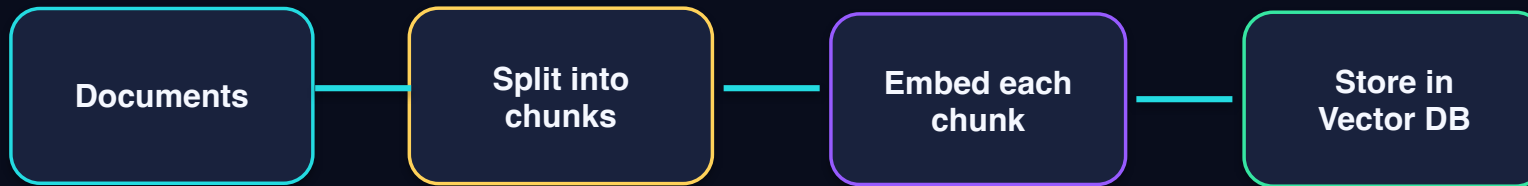
DEFAULT STARTUP MOVE

Start with RAG + evals. Fine-tune only when you can prove the bottleneck is model behavior.

Production RAG architecture

A startup-friendly reference pattern you can build incrementally.

Build like this



Deploy like this



The hidden half of RAG: ingestion

Most quality gains happen before the user asks the question.

1

Connect

PDFs, web, Slack, DB

2

Clean

remove boilerplate

3

Chunk

semantic boundaries

4

Embed

text → vector

5

Index

metadata + ACLs

Store source URL, owner, timestamp, permissions, and content hash from day one.

Chunking is product design

A chunk should be big enough to answer, small enough to retrieve.

Raw document

Pricing policy

Starter includes 1,000 credits.
Pro includes 5,000 credits/month.
Enterprise has custom limits.
Refunds within 14 days if usage is below 100 credits.

Chunk A
pricing + starter

overlap

Chunk B
pro credits

overlap

Chunk C
refund rule

Bad chunking = correct answer hidden in the wrong box.

Semantic Similarity

Similar ideas land near each other in vector space.

Embeddings meaning

- "cheap hotel in London"
"find planet close to earth"

→ [0.13, -0.82, 0.44, 0.09, ...]

- "budget accommodation in UK capital"
"The closest planet to earth is ..."

→ [0.15, -0.79, 0.41, 0.11, ...]

Cosine-sim = 0.96

Cosine similarity

cosine similarity = 1 => similar

cosine similarity = 0 => unrelated

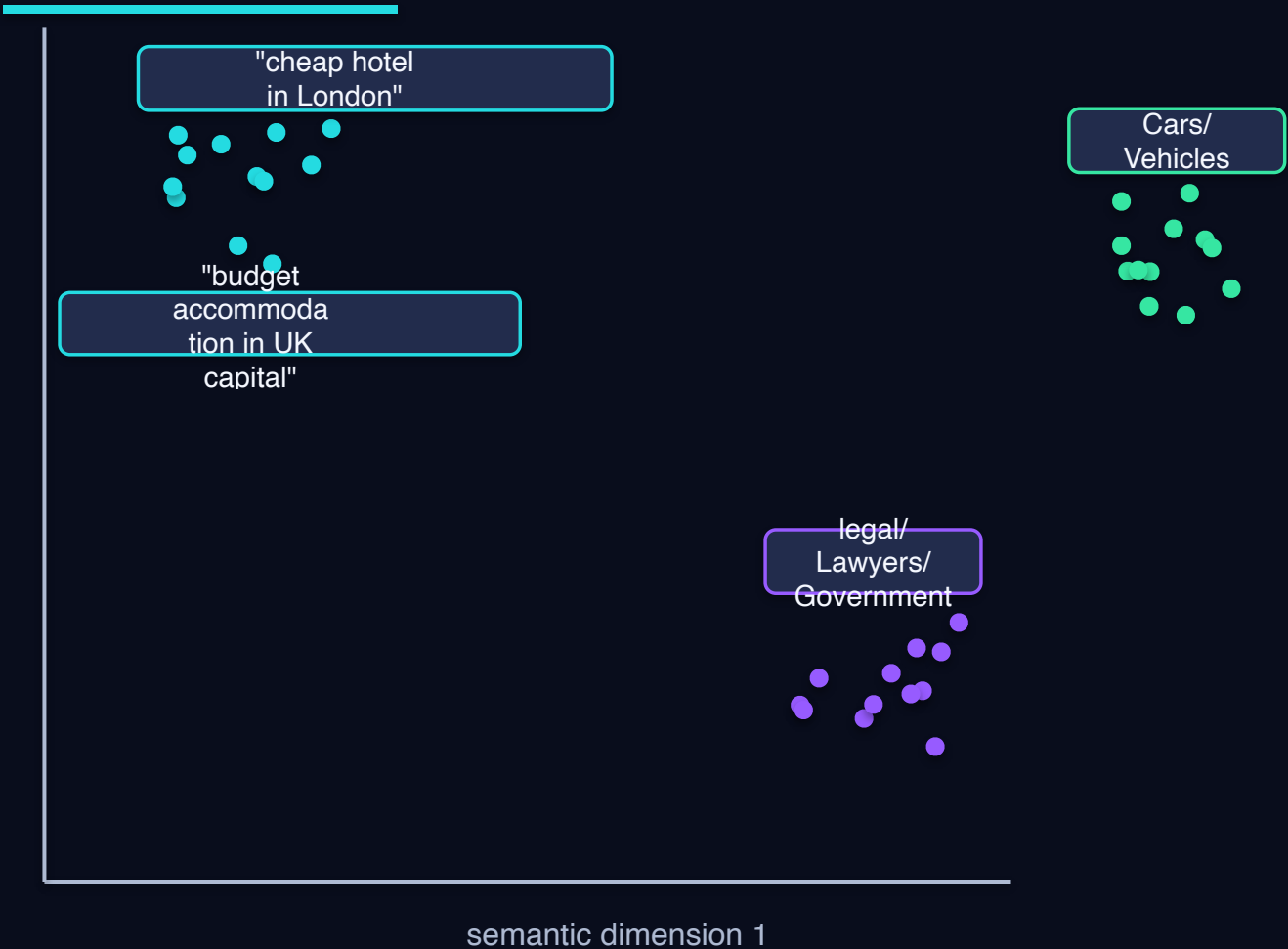
cosine similarity = -1 => opposite

I love this product
I hate this product

$$\text{cos_sim}(A, B) = A \cdot B / (\|A\| \times \|B\|)$$

Semantic Similarity

Similar ideas land near each other in vector space.



Startup use cases that pay

RAG is strongest when answers come from a changing knowledge base.

1

Support copilot

deflect tickets + faster agents

2

Sales assistant

answers from decks, CRM, docs

3

Legal/doc review

contract facts + citations

4

Internal search

Slack, Notion, Jira, Drive

5

Onboarding tutor

role-specific company knowledge

6

Codebase assistant

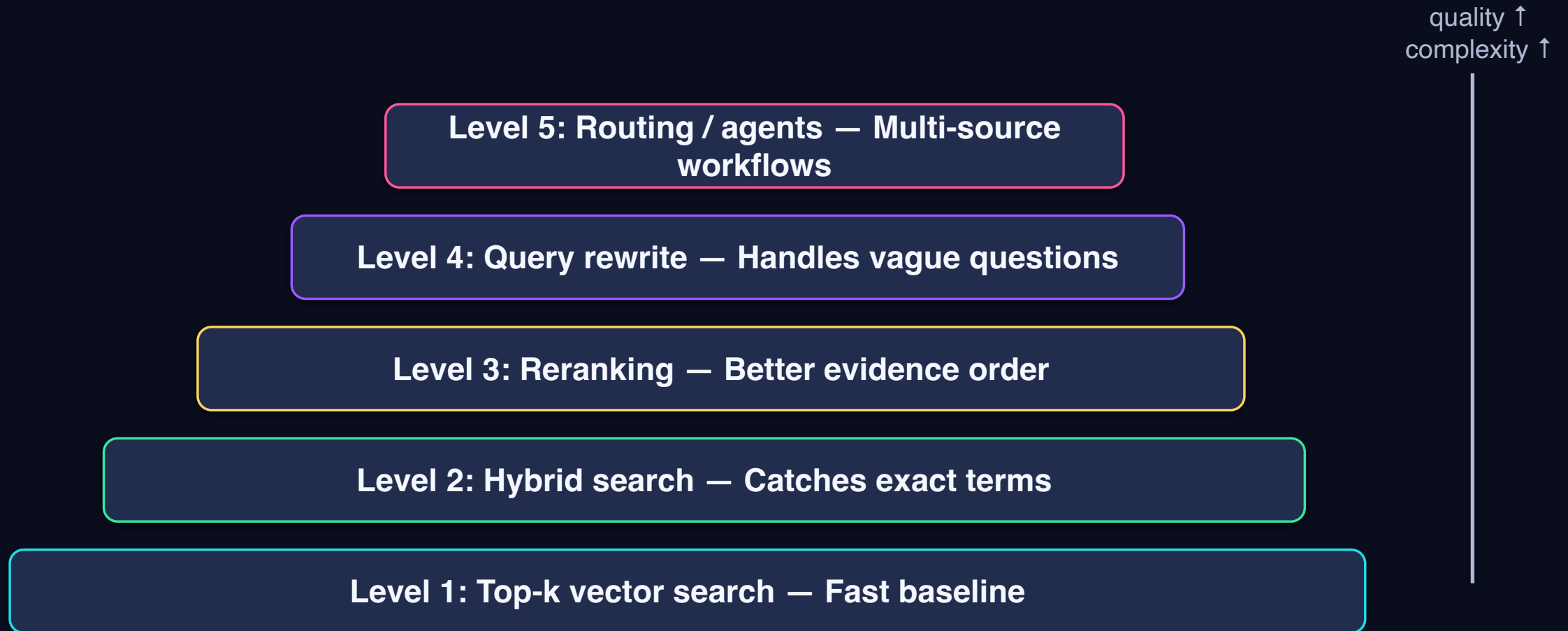
repo + docs + issues

AUDIENCE PICK

Which one would you build first with a 2-week MVP budget?

Retrieval quality ladder

Each level increases quality, but also complexity and latency.

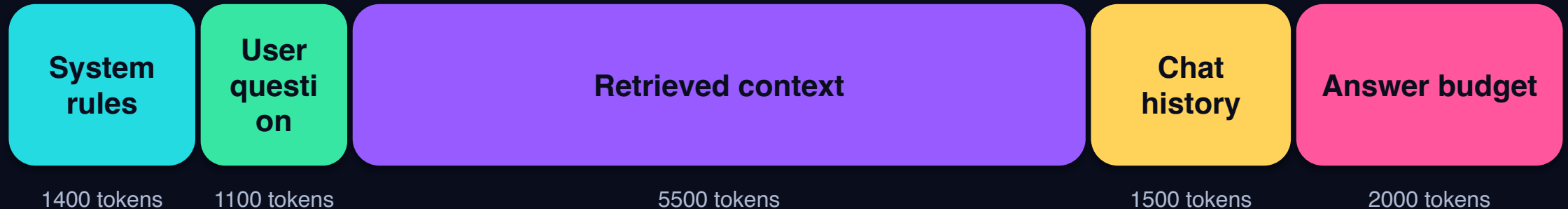


DISCUSSION

Where would you stop for an MVP?

Prompt assembly: context window is a budget

Every token competes for space.



Practical rule: pass fewer, higher-quality chunks. More context can create more confusion.

MVP checklist: max context size, source count, citation requirement, refusal behavior.

You cannot improve what you cannot measure.

You cannot improve what you cannot measure.



Common RAG failure modes

Debug systematically: retrieval, context, generation, product UX.

No answer in corpus

The data simply does not exist

Add source /
refusal

Wrong chunk retrieved

Embedding or chunking issue

Hybrid + rerank

Stale answer

Old source ranks higher

Freshness
metadata

Prompt injection

Doc tells model to ignore rules

Sanitize + isolate

No trust

Answer has no evidence

Citations + trace UI

Too slow

Too many calls / reranking

Cache + budget

Security and trust

The boring part that saves the company.

User auth

Document ACLs

PII redaction

Prompt injection defense

Citations + audit logs

Product rule: if a human user cannot see a document, the retriever must not retrieve it.

2-week project

Prove one painful workflow, not a universal knowledge brain.

1

Days 1–2

Pick niche + gather
50 real questions

2

Days 3–4

Ingest 3–5 data
sources with
metadata

3

Days 5–7

Baseline RAG +
citations UI

4

Days 8–10

Eval set + failure
tagging

5

Days 11–14

Rerank, guardrails,
pilot users

SUCCESS METRIC

Can 5 pilot users solve a real task faster with fewer wrong answers?

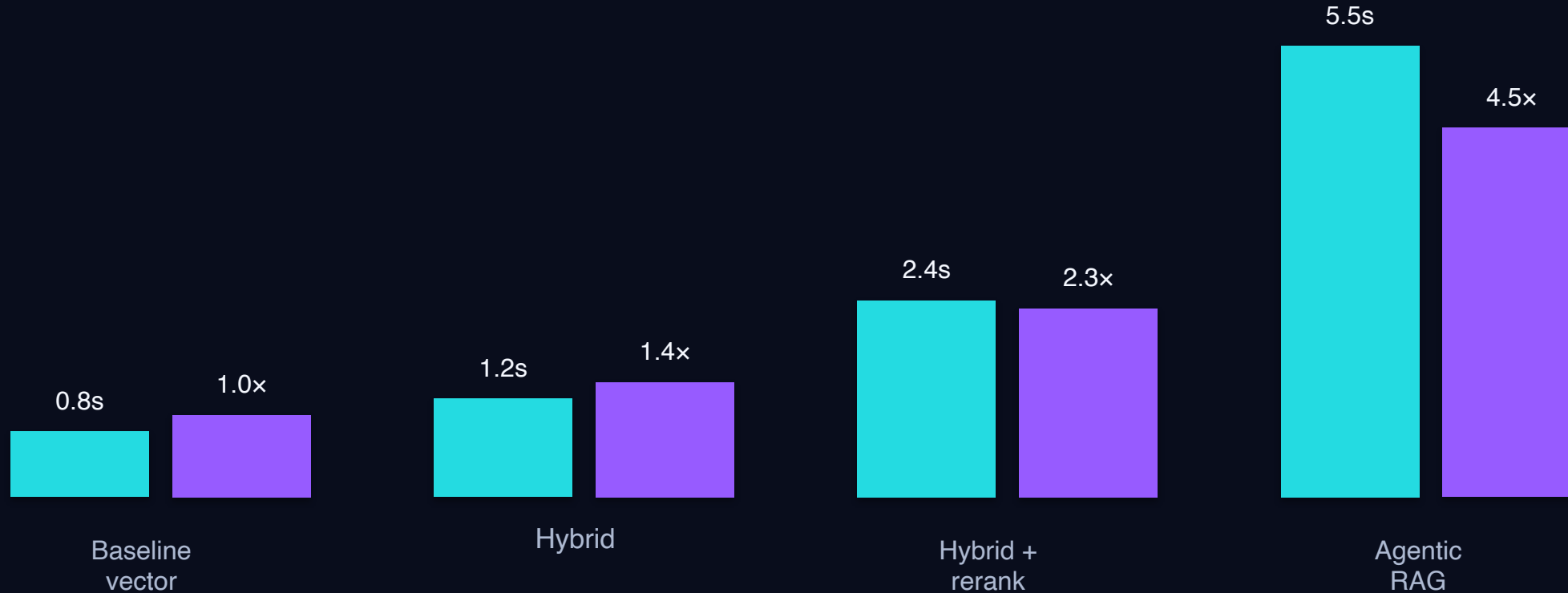
Cost and latency economics

RAG quality is a trade-off; product teams need explicit budgets.

Illustrative trade-off chart, not benchmark data.

Latency

Cost



Set budgets early: max latency, max cost per answer, max source count, and quality threshold.

Live demo script: support copilot

A practical flow you can run in class or with a startup team.

1

Upload pricing FAQ + refund policy

2

Ask: “Can I get a refund after using 200 credits?”

3

Show retrieved chunks + citations

4

Break it: ask vague / malicious / outdated question

5

Fix it: metadata, rerank, refusal, logs

DEMO TWIST

The goal is not magic. The goal is to expose the debugging loop.

Interactive workshop: design your startup RAG

Groups get 8 minutes, then pitch the architecture in 60 seconds.

Data

What sources are needed?
Who owns them?

User job

What painful workflow changes?

Retriever

Keyword, vector, hybrid, rerank?

Trust

How will users verify the answer?

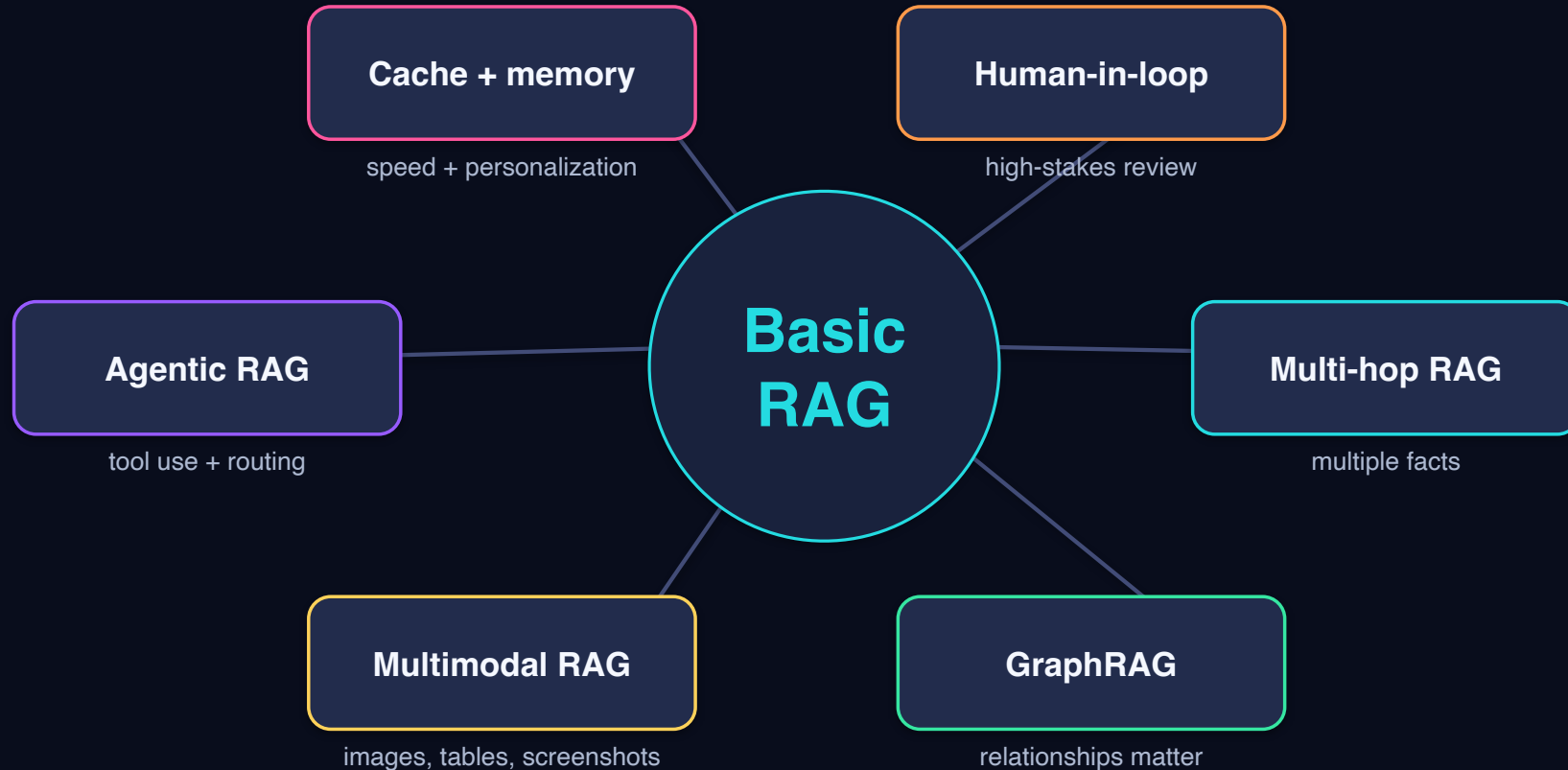
Metric

What proves this is worth paying for?

**Pitch: problem → data →
flow → metric**

Beyond basic RAG

Add complexity only when the product demands it.



Final checklist before shipping RAG

Use this as your production readiness gate.

- ✔ Can we cite the source?
- ✔ Do permissions work?
- ✔ Do we know when to refuse?
- ✔ Do we log failures?
- ✔ Can we measure answer quality?
- ✔ Is cost/latency within budget?
- ✔ Can non-technical users trust it?
- ✔ Is the MVP solving one clear job?